34.3 链接理解与媒体理解

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~82k tokens，输出 ~9k tokens（本节）

当用户在聊天中发送一个 YouTube 链接，或者转发一段语音消息、一张照片，纯文本 Agent 是无法“看到”或“听到”这些内容的。OpenClaw 通过两个理解系统弥补这一鸿沟：**链接理解**（Link Understanding）将 URL 背后的内容提取为文本摘要，**媒体理解**（Media Understanding）将图片、音频、视频转换为文字描述或转录文本，最终将这些附加信息注入到消息上下文中，让 Agent 能够“理解”多媒体内容。

34.3.1 链接理解（`src/link-understanding/`）

架构概览

链接理解模块是一个轻量级的管道，核心流程如下：

```
用户消息 → extractLinksFromMessage（URL 提取）
  → resolveScopeDecision（作用域检查）
    → runLinkEntries（CLI 工具链执行）
      → formatLinkUnderstandingBody（结果注入消息体）
```

URL 提取

`extractLinksFromMessage` 从用户消息中提取裸 URL（即非 Markdown 链接语法内的 URL）：

```ts
// src/link-understanding/detect.ts
const MARKDOWN_LINK_RE = /\[[^\]]*]\((https?:\/\/\S+?)\)/gi;
const BARE_LINK_RE = /https?:\/\/\S+/gi;

export function extractLinksFromMessage(message: string, opts?: { maxLinks?: number }):
  // 1. 先移除 Markdown 链接 [text](url)，避免重复提取
  const sanitized = stripMarkdownLinks(message);

  // 2. 用正则匹配所有裸 URL
  const seen = new Set<string>();      // 去重
  const results: string[] = [];
  for (const match of sanitized.matchAll(BARE_LINK_RE)) {
    if (!isAllowedUrl(match[0])) continue;  // 过滤 127.0.0.1 等内部地址
    if (seen.has(match[0])) continue;
    results.push(match[0]);
    if (results.length >= maxLinks) break;   // 默认最多 3 个
  }
  return results;
}
```

设计要点：先去除 Markdown 链接语法的原因是，`[点击这里](https://example.com)` 中的 URL 是用户有意隐藏在文本背后的引用，不应被当作需要理解的独立链接；而 `请看 https://example.com` 中的裸 URL 才是用户希望 Agent 理解的内容。

CLI 工具链驱动

链接理解不内置任何网页解析逻辑，而是通过配置的外部 CLI 工具来执行。每个工具条目（`LinkModelConfig`）定义了命令和参数模板：

```yaml
// 配置示例（YAML）
tools:
  links:
    enabled: true
    maxLinks: 3
    models:
      - type: cli
        command: "node"
        args: ["./scripts/fetch-url.js", "{{LinkUrl}}"]
        timeoutSeconds: 30
```

`runCliEntry` 将 URL 注入模板变量 `{{LinkUrl}}`，然后通过 `runExec` 执行子进程。模板引擎复用了第 10 章介绍的 `applyTemplate` 函数，支持消息上下文中的所有变量。

多个工具条目按顺序尝试（fallback 链）：

```
// src/link-understanding/runner.ts
async function runLinkEntries(params): Promise<string | null> {
  for (const entry of params.entries) {
    try {
      const output = await runCliEntry({ entry, url, ctx, config });
      if (output) return output;  // 第一个有输出的即返回
    } catch (err) {
      // 记录失败，尝试下一个
    }
  }
  return null;  // 所有工具都失败
}
```

作用域控制

链接理解复用了媒体理解模块的作用域系统（`resolveMediaUnderstandingScope`），支持按通道（channel）、聊天类型（chatType）、会话前缀（keyPrefix）来控制启用/禁用：

```
# 只在 Telegram 私聊中启用链接理解
tools:
  links:
    scope:
      default: deny
      rules:
        - match: { channel: telegram, chatType: private }
          action: allow
```

结果注入

当 CLI 工具返回了链接内容摘要，`applyLinkUnderstanding` 将结果追加到消息体末尾：

```
// src/link-understanding/format.ts
export function formatLinkUnderstandingBody(params): string {
  const base = (params.body ?? "").trim();
  if (!base) return outputs.join("\n");
  return `${base}\n\n${outputs.join("\n")}`;
}
```

这样 Agent 收到的消息变为：

```
用户原始文本

[链接1的内容摘要]
[链接2的内容摘要]
```

34.3.2 媒体理解（`src/media-understanding/`）

架构概览

媒体理解模块的规模和复杂度远超链接理解。它处理三种媒体能力：图像描述（image.description）、音频转录（audio.transcription）、视频描述（video.description），以及附带的文件内容提取。整体架构分为四层：

```
                    ┌──────────────────────────┐
                    │   applyMediaUnderstanding│  ← 入口编排层
                    │   (apply.ts)             │
                    └──────────────────────────┘
                              │
                              │
              ┌───────────────┼───────────────┐
              ▼               ▼               ▼
         runCapability   runCapability  runCapability    ← 能力执行层
         (image)         (audio)        (video)
              │               │               │
              ▼               ▼               ▼
        ┌──────────────────────────────────────┐
        │       Provider / CLI 调度层          │         ← 模型调度层
        │  runProviderEntry / runCliEntry      │
        └──────────────────────────────────────┘
              │               │               │
              ▼               ▼               ▼
        ┌─────────┐   ┌──────────┐   ┌─────────┐
        │ OpenAI  │   │ Deepgram │   │ Google  │  ...      ← Provider 实现层
        │ Groq    │   │ Anthropic│   │ MiniMax │
        └─────────┘   └──────────┘   └─────────┘
```

类型系统

```typescript
// src/media-understanding/types.ts
export type MediaUnderstandingKind =
  | "audio.transcription"    // 音频转录为文字
  | "video.description"      // 视频内容描述
  | "image.description";     // 图像内容描述

export type MediaUnderstandingCapability = "image" | "audio" | "video";

export type MediaUnderstandingProvider = {
  id: string;
  capabilities?: MediaUnderstandingCapability[];
  transcribeAudio?: (req: AudioTranscriptionRequest) => Promise<AudioTranscriptionResul
  describeVideo?: (req: VideoDescriptionRequest) => Promise<VideoDescriptionResult>;
  describeImage?: (req: ImageDescriptionRequest) => Promise<ImageDescriptionResult>;
};
```

每个 Provider 可以实现一个或多个能力接口。比如 OpenAI 同时支持音频转录（Whisper API）和图像描述（GPT-4o Vision），而 Deepgram 只支持音频转录。

Provider 注册表

六个内置 Provider 在 `providers/index.ts` 中注册：

Provider	音频	图像	视频	默认模型
Groq	✓	-	-	whisper-large-v3-turbo
OpenAI	✓	✓	-	gpt-4o-mini-transcribe
Google	-	✓	✓	gemini-3-flash-preview
Anthropic	-	✓	-	claude-opus-4-6
MiniMax	-	✓	-	MiniMax-VL-01
Deepgram	✓	-	-	nova-3

自动模型发现

当用户没有在配置文件中显式指定媒体理解模型时，系统通过 `resolveAutoEntries` 自动发现可用的模型，搜索顺序为：

```
1. 当前 Agent 正在使用的主模型（如果支持该能力）
2. 本地 CLI 工具（音频: sherpa-onnx → whisper-cli → whisper）
3. Gemini CLI（如果 `gemini` 命令可用）
4. 按优先级探测云端 API Key：
   - 音频: openai → groq → deepgram → google
   - 图像: openai → anthropic → google → minimax
   - 视频: google
```

> **衍生解释 — Whisper**
>
> Whisper 是 OpenAI 开源的自动语音识别（ASR）模型，支持多语言转录和翻译。 `whisper-cli` 是其 C++ 移植版（whisper.cpp），运行在本地 CPU 上无需 GPU。OpenClaw 优先探测本地 Whisper 安装，避免 API 调用开销。

智能跳过机制

一个巧妙的优化：当 Agent 使用的主模型（如 GPT-4o）本身支持视觉能力时， `runCapability` 会跳过图像描述——因为图像会直接作为多模态输入传递给主模型，无需额外的描述步骤：

```
// src/media-understanding/runner.ts（简化）
if (capability === "image" && activeProvider) {
  const entry = findModelInCatalog(catalog, activeProvider, model);
  if (modelSupportsVision(entry)) {
    // 跳过：主模型原生支持视觉，图像将直接注入模型上下文
    return { outputs: [], decision: { outcome: "skipped" } };
  }
}
```

附件缓存

`MediaAttachmentCache` 类缓存了每个附件的下载结果，避免同一附件被多个能力重复下载：

```
export class MediaAttachmentCache {
  private readonly entries = new Map<number, AttachmentCacheEntry>();

  // 获取 Buffer（从本地路径读取 或 从 URL 下载）
  async getBuffer(params: { attachmentIndex, maxBytes, timeoutMs }): Promise<MediaBuffe

  // 获取文件路径（如果是远程 URL，先下载到临时文件）
  async getPath(params: { attachmentIndex, maxBytes, timeoutMs }): Promise<MediaPathRes

  // 清理所有临时文件
  async cleanup(): Promise<void>
}
```

CLI 工具需要文件路径（ `getPath` ），API Provider 需要 Buffer（ `getBuffer` ）。缓存层统一处理了这两种访问模式，并在最后统一清理临时文件。

并发控制

媒体理解涉及网络 I/O（下载附件、调用 API），耗时可能较长。 `runWithConcurrency` 实现了一个简单但有效的工人池模型：

```ts
// src/media-understanding/concurrency.ts
export async function runWithConcurrency<T>(
  tasks: Array<() => Promise<T>>,
  limit: number,
): Promise<T[]> {
  const results = Array.from({ length: tasks.length });
  let next = 0;
  const workers = Array.from({ length: limit }, async () => {
    while (true) {
      const index = next++;
      if (index >= tasks.length) return;
      results[index] = await tasks[index]();
    }
  });
  await Promise.allSettled(workers);
  return results;
}
```

默认并发度为 2（`DEFAULT_MEDIA_CONCURRENCY`），确保图像、音频、视频三种能力最多有两个并发执行。

作用域规则系统

媒体理解通过规则链控制不同场景下的启用状态：

```ts
// src/media-understanding/scope.ts
export function resolveMediaUnderstandingScope(params: {
  scope?: MediaUnderstandingScopeConfig;
  sessionKey?: string;
  channel?: string;
  chatType?: string;
}): "allow" | "deny" {
  for (const rule of scope.rules) {
    // 规则匹配：channel + chatType + keyPrefix 三维度
    if (matchChannel && matchChannel !== channel) continue;
    if (matchChatType && matchChatType !== chatType) continue;
    if (matchPrefix && !sessionKey.startsWith(matchPrefix)) continue;
    return rule.action;  // 第一个匹配的规则决定结果
  }
  return scope.default ?? "allow";  // 未匹配则使用默认策略
}
```

CLI 输出解析

当使用 CLI 工具（如 `whisper-cli` 、 `gemini` 、 `sherpa-onnx-offline` ）时，不同工具的输出格式各异。 `resolveCliOutput` 实现了多策略的输出解析：

工具	解析策略
`whisper-cli`	读取 `{outputBase}.txt` 文件
`whisper`	读取 `{outputDir}/{basename}.txt` 文件
`gemini`	从 JSON 标准输出中提取 `response` 字段
`sherpa-onnx-offline`	从 JSON 中递归提取 `text` 字段
其他	直接使用标准输出文本

入口编排

`applyMediaUnderstanding` 是整个模块的入口，编排了完整的媒体理解流程：

```
// src/media-understanding/apply.ts（流程简化）
export async function applyMediaUnderstanding(params): Promise<ApplyMediaUnderstandingR
  const attachments = normalizeMediaAttachments(ctx);
  const cache = createMediaAttachmentCache(attachments);

  try {
    // 1. 并发执行三种能力（image / audio / video）
    const tasks = ["image", "audio", "video"].map(capability =>
      () => runCapability({ capability, cfg, ctx, attachments: cache, ... })
    );
    const results = await runWithConcurrency(tasks, concurrency);

    // 2. 将理解结果注入消息上下文
    if (outputs.length > 0) {
      ctx.Body = formatMediaUnderstandingBody({ body: ctx.Body, outputs });
      // 音频转录替换原始消息体（语音消息场景）
      if (audioOutputs.length > 0) {
        ctx.Transcript = formatAudioTranscripts(audioOutputs);
        ctx.CommandBody = originalUserText ?? transcript;
      }
    }

    // 3. 提取文件内容块（PDF、文本文件等）
    const fileBlocks = await extractFileBlocks({
      attachments, cache, limits: resolveFileLimits(cfg),
      skipAttachmentIndexes: audioAttachmentIndexes,  // 已转录的音频不重复处理
    });
    if (fileBlocks.length > 0) {
      ctx.Body = appendFileBlocks(ctx.Body, fileBlocks);
    }

    // 4. 刷新入站上下文
    finalizeInboundContext(ctx, { forceBodyForAgent: true });
  } finally {
    await cache.cleanup();  // 清理临时文件
  }
}
```

文件内容提取

除图像/音频/视频外，用户发送的文本类附件（PDF、CSV、JSON、Markdown 等）通过 `extractFileBlocks` 提取内容。该函数的处理逻辑包含大量编码检测：

1. **UTF-16 BOM 检测**：通过字节序标记（BOM）识别 UTF-16LE/BE 编码
2. **统计检测**：对没有 BOM 的文件，通过统计零字节的奇偶位置分布推断 UTF-16 编码
3. **Legacy 编码回退**：对非 UTF-8 的西文文件，通过 CP1252 映射表解码（Windows 默认编码）
4. **CSV/TSV 猜测**：根据第一行的逗号和制表符数量推断分隔符格式

提取后的内容包装为 XML 块注入消息体：

```
<file name="report.pdf" mime="application/pdf">
提取到的文本内容...
</file>
```

文件名和 MIME 类型经过 XML 转义处理，防止属性值注入攻击。

决策追踪

每次媒体理解都会生成一个 `MediaUnderstandingDecision` 结构，记录每个附件尝试了哪些模型、成功/跳过/失败的原因。这些决策信息存储在消息上下文的 `MediaUnderstandingDecisions` 字段中，用于调试和监控：

```typescript
export type MediaUnderstandingDecision = {
  capability: "image" | "audio" | "video";
  outcome: "success" | "skipped" | "disabled" | "no-attachment" | "scope-deny";
  attachments: Array<{
    attachmentIndex: number;
    attempts: Array<{ provider, model, type, outcome, reason }>;
    chosen?: { provider, model, outcome };
  }>;
};
```

本节小结

1. **链接理解**从消息中提取裸 URL，通过可配置的 CLI 工具链获取内容摘要，并将结果追加到消息体中。
2. **媒体理解**支持图像描述、音频转录、视频描述三种能力，通过 Provider 接口层统一六个内置后端（Groq、OpenAI、Google、Anthropic、MiniMax、Deepgram）。
3. **自动模型发现**在无显式配置时，按优先级探测本地 CLI 工具和云端 API Key，选择最佳可用模型。
4. **智能跳过**在主模型已支持视觉时跳过图像描述，避免冗余处理。

5. **附件缓存**统一管理 Buffer 和文件路径的获取，CLI 和 API 两种消费模式共享下载结果。
6. **作用域规则**支持按通道、聊天类型、会话前缀三维度控制启用策略。
7. **文件内容提取**处理 PDF、CSV、文本等非媒体附件，包含 UTF-16、CP1252 等复杂编码检测。
8. **决策追踪**完整记录每次理解的模型选择、成功/失败原因，便于调试和监控。